

Algoritmica grafurilor - Cursul 2

9 ottobre 2020

1 Vocabularul teoriei grafurilor

- Variații în definiția unui graf
- Grade
- Subgrafuri
- Operații cu grafuri
- Clase de grafuri
- Drumuri și circuite

2 Exerciții pentru seminarul de săptămâna viitoare

3 Exerciții rezolvate (parțial)

Variații în definiția unui graf

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Multigraf: $G = (V, E)$, unde V este o mulțime nevidă (de noduri), și E este un **multiset** (de muchii) pe V , i. e., există o funcție $m : \binom{V}{2} \rightarrow \mathbb{N}$.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

$e \in \binom{V}{2}$, cu $m(e) > 0$ este o muchie a multigrafului G ; dacă $m(e) = 1$, atunci e este o **muchie simplă**, altfel este o **muchie multiplă** cu **multipllicitatea** $m(e)$.

Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Graful suport al unui graf, G , este graful obținut din G prin înlocuirea fiecărei muchii multiple printr-una simplă.

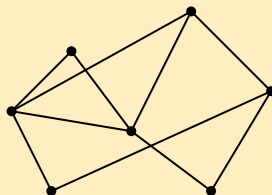
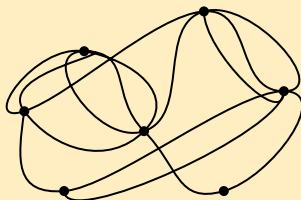
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Variații în definiția unui graf

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.

Exemplu

Un multigraf și graful său suport:



C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Variații în definiția unui graf

Pseudograf (graf general): $G = (V, E)$, unde V este o mulțime (de noduri), și E este un **multiset** (de muchii) peste $V \cup \binom{V}{2}$, i. e., există o funcție $m : V \cup \binom{V}{2} \rightarrow \mathbb{N}$.

$e \in E \cap V$ (i. e., $|e| = 1$) este numită **buclă**.

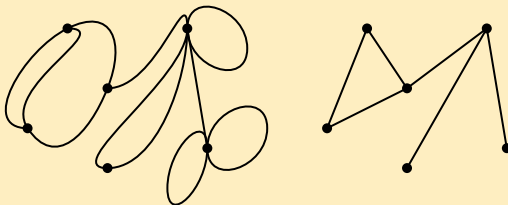
Graful suport al unui pseudograf G este graful obținut din G prin înlocuirea fiecărei muchii multiple printr-una simplă și prin ștergerea buclelor.

Variații în definiția unui graf

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.

Exemplu

Un pseudograf și graful său suport:



C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Variații în definiția unui graf

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Digraf: $D = (V(D), E(D))$, unde $V(D)$ este o mulțime (de noduri), și $E(D) \subseteq V(D) \times V(D)$ este o mulțime de **arce** (sau **muchii orientate**).

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Dacă $e \in E$ atunci $e = (u, v)$ (sau simplu $e = uv$) este un arc orientat de la u către v și spunem că:

- u este extremitatea inițială of e , v este extremitatea finală ale lui e ;
- u și v sunt adiacente;
- e este incident din u și către v ;
- v este a succesor al lui u și u este a predecesor al lui v etc.

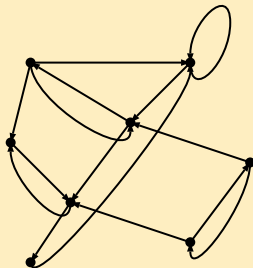
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Variații în definiția unui graf

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Exemplu

Un digraf:



C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Variații în definiția unui graf

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

- **Pereche simetrică de arce:** (uv, vu) . uv este numit inversul lui vu .
- **Inversul** unui digraf D : se înlocuiește fiecare arc din D cu inversul său.
- **Graful suport** al unui digraf D , $M(D)$, se înlocuiește fiecare arc din D cu mulțimea corespunzătoare de două noduri. $M(D)$ este un multigraf.
- Dacă $M(D)$ este un graf (simplu), atunci D este numit **graf orientat**.
- **Digraf complet simetric:** orice două noduri (distincte) sunt unite printr-o pereche simetrică de arce.
- **Turneu:** un graf orientat complet (orice două noduri (distincte) sunt unite exact printr-un arc).

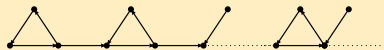
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Variații în definiția unui graf

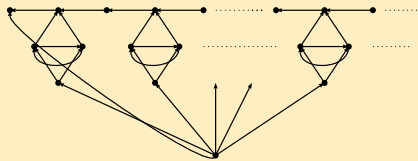
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

(Di)grafuri infinite: mulțimea nodurilor și/sau mulțimea muchiilor (arcelor) este numărabil infinită.

Un graf infinit este **local finit** dacă $N(v)$ este o mulțime finită, pentru orice nod v .



The Danaids barrel



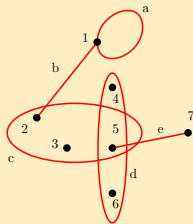
The sisyphus graph

- Graph Algorithms - C. Croitoru - Graph Algorithms - C. Croitoru - Graph Algorithms

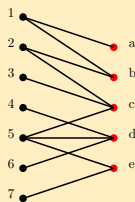
Hipergrafuri (Sisteme de mulțimi finite)

- Muchiile, numite acum **hipermuchi**, nu mai sunt restricționate să fie submulțimi cu două elemente ale mulțimii de noduri. O hipermuchie este submulțime a mulțimii de noduri.
- **Hipergrafuri k -uniforme**: fiecare muchie are cardinalul k .

Fiecare hipergraf poate fi reprezentat ca un graf bipartit:



Hypergraph H



Bipartite graph associated with H

Fie $G = (V, E)$ un graf și $v \in V$.

- **Gradul** unui nod v : $d_G(v)$ = numărul de muchii incidente cu v .
- v este un **nod izolat** dacă $d_G(v) = 0$ și **pendant** (sau **frunză**) dacă $d_G(v) = 1$.

$$\sum_{v \in V} d_G(v) = 2|E|.$$

- **Gradul maxim** $\Delta(G)$ și **gradul minim** $\delta(G)$:

$$\Delta(G) = \max_{v \in V} d_G(v), \quad \delta(G) = \min_{v \in V} d_G(v).$$

- Dacă $\Delta(G) = \delta(G) = k$, atunci G este **k -regulat**.
- **Graf nul**: un graf 0-regulat .

Fie $G = (V, E)$ un digraf și $v \in V$.

- Gradul interior al unui nod v : $d_G^-(v)$ = numărul de arce incidente spre v .
- Gradul exterior al unui nod v : $d_G^+(v)$ = numărul de arce incidente din v .

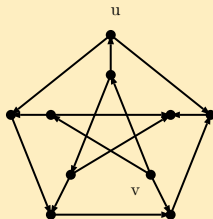
$$\sum_{v \in V} d_G^+(v) = \sum_{v \in V} d_G^-(v) = |E|.$$

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
 * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
 Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
 Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
 - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.

Exemplu

$$d_G^+(u) = 2, d_G^-(u) = 1; d_G^+(v) = 3, d_G^-(v) = 0$$



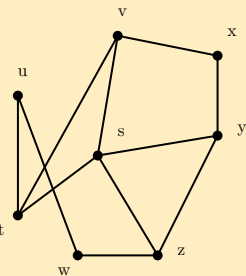
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
 * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
 Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
 Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
 - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Fie $G = (V(G), E(G))$ un graf.

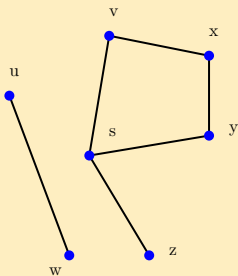
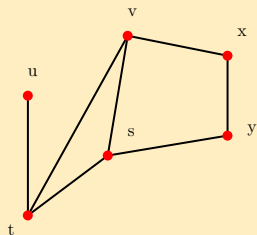
- **Subgraf** al lui G : un graf $H = (V(H), E(H))$ așa încât $V(H) \subseteq V(G)$ și $E(H) \subseteq E(G)$.
- **Graf parțial** al lui G : un subgraf H al lui G astfel ca $V(H) = V(G)$.
- **Subgraf generat de** $B \subseteq E(G)$ **în** G : un subgraf al lui G , $H = (V(H), E(H))$, astfel că $E(H) = B$ și $V(H) = \cup_{uv \in B} \{u, v\}$. Se notează prin $\langle B \rangle_G$.
- **Subgraf indus**: un subgraf H al lui G așa încât $E(H) = \binom{V(H)}{2} \cap E(G)$. Dacă $A \subseteq V(G)$, **subgraful indus de** A **în** G este notat cu $[A]_G$ sau $G[A]$.

Algorithm * C Custom, C++ Custom, C++ Custom, C++ Custom, C++ Custom

comple



G

 H'  H''

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Fie $G = (V(G), E(G))$ un graf.

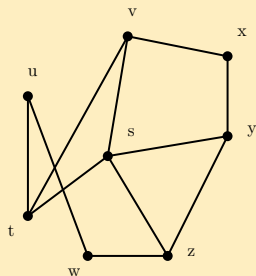
- Dacă $A \subseteq V(G)$, atunci subgraful $[V(G) \setminus A]_G$, notat prin $G - A$, este subgraful obținut din G prin ștergerea nodurilor lui A . $G - \{u\}$ este numit **subgraf de ștergere** și se notează, simplu cu $G - u$.
- Dacă $B \subseteq E(G)$, atunci subgraful $\langle E(G) \setminus B \rangle_G$, notat prin $G - B$, este subgraful obținut din G prin ștergerea tuturor muchiilor lui B . $G - \{e\}$ se notează $G - e$.
- Definiții și notații similare pentru digrafuri, multigrafuri etc.

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

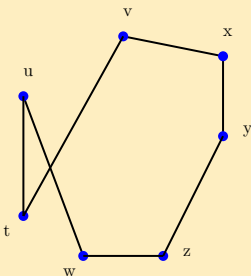
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exemplu

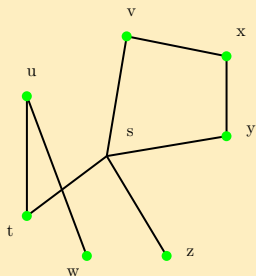
Un graf G , $G - s$, și $G - \{vt, wz, zy\}$.



G



$G - s$



$G - \{vt, wz, zy\}$

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Operații cu grafuri

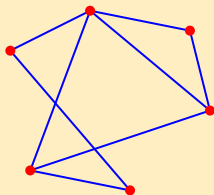
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Operație unară: $G = (V(G), E(G))$

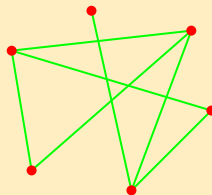
- **Complementul** unui graf G : un graf $\overline{G}$, cu $V(\overline{G}) = V(G)$ și

$$E(\overline{G}) = \binom{V(G)}{2} \setminus E(G).$$

Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *



G



$\overline{G}$

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

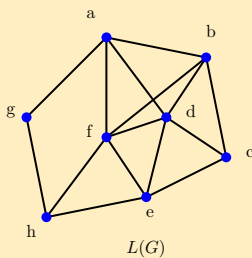
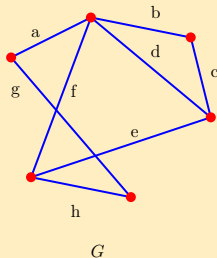
Operații cu grafuri

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Operație unară: $G = (V(G), E(G))$

- **Graful reprezentativ al muchiilor (line-graful)** lui G : un graf $L(G)$, cu $V(L(G)) = E(G)$ și

$$E(L(G)) = \{ef : e, f \in E(G), e \text{ și } f \text{ sunt adiacente în } G\}.$$



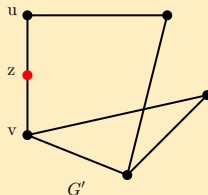
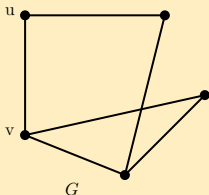
Operații cu grafuri

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Operație unară: $G = (V(G), E(G))$

- **Graful obținut din G prin inserarea unui nod nou (z) pe o muchie ($e = uv$):** graful G' , cu $V(G') = V(G) \cup \{z\}$ și

$$E(G') = E(G) \setminus \{uv\} \cup \{uz, zv\}.$$



Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Operații cu grafuri

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

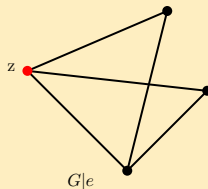
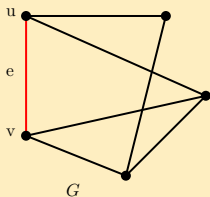
Operație unară: $G = (V(G), E(G))$

- **Graful obținut din G prin contracția unei muchii $e = uv \in E(G)$:** graful $G|e$ cu

$$V(G|e) = V(G) \setminus \{u, v\} \cup \{z\},$$

$$E(G|e) = E([V(G) \setminus \{u, v\}]_G) \cup \{yz : yu \text{ sau } yv \in E(G)\}.$$

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph



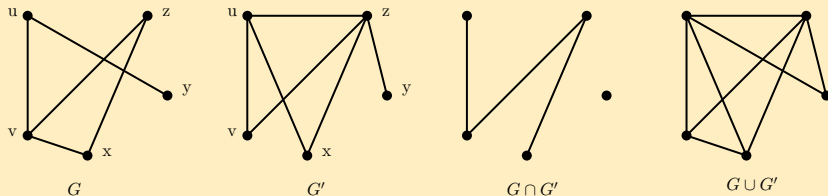
Operații cu grafuri

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Operații binare: G, G' cu $V(G) = V(G')$

- **Intersecția** $G \cap G' = (V(G), E(G) \cap E(G'))$.
- **Reuniunea** $G \cup G' = (V(G), E(G) \cup E(G'))$.

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.



Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

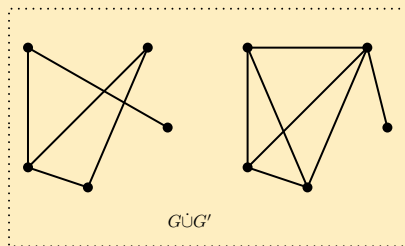
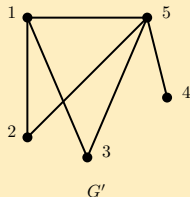
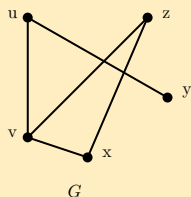
Operații cu grafuri

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Operație binară: G, G' cu $V(G) \cap V(G') = \emptyset$

- **Reuniunea disjunctă** $G \dot{\cup} G' = (V(G) \cup V(G'), E(G) \cup E(G'))$.

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

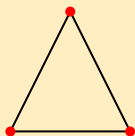


Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

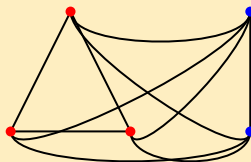
Operații cu grafuri

Operație binară: G, G' cu $V(G) \cap V(G') = \emptyset$

- **Suma directă (join)** $G + G' = \overline{\overline{G} \dot{\cup} \overline{G'}}$.



G

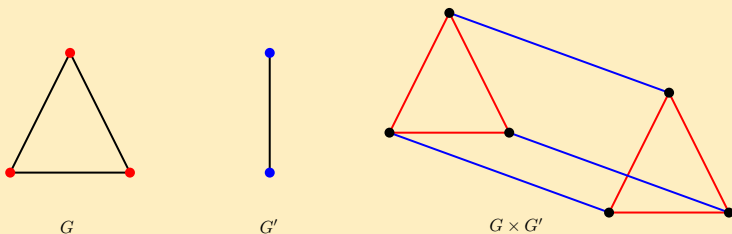
 G^M 
$$G + G'$$
[illegible]

Operație binară: G, G' cu $V(G) \cap V(G') = \emptyset$

- **Produsul cartezian** al grafurilor G și G' : graful $G \times G'$ cu

$$V(G \times G') = V(G) \times V(G').$$

$$E(G \times G') = \{(u, u')(v, v') : u, v \in V(G), u', v' \in V(G'), \\ u = v \text{ și } u'v' \in E(G') \text{ sau } u' = v' \text{ și } uv \in E(G)\}.$$



Clase de grafuri - Grafurile complete

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.

Graful complet de ordin n , K_n : $|V(K_n)| = n$ și $E(K_n) = \binom{V(K_n)}{2}$.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru



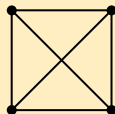
K_1



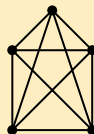
K_2



K_3



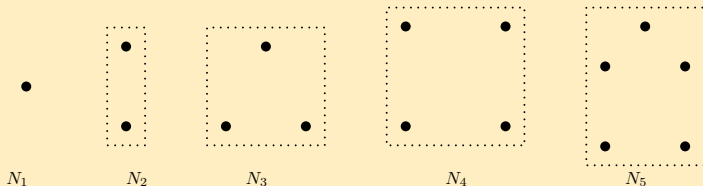
K_4



K_5

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Graful nul de ordin n , N_n : $|V(K_n)| = n$ și $E(K_n) = \emptyset$.



Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Clase de grafuri - Circuitele C_n

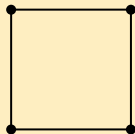
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Circuitul de ordin n , C_n : $V(C_n) = \{1, 2, \dots, n\}$ și $E(C_n) = \{12, 23, \dots, n-1n, n1\}$.

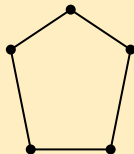
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -



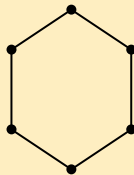
C_3



C_4



C_5



C_6

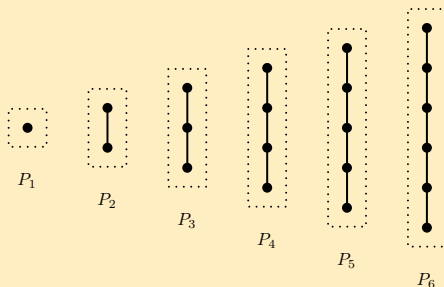
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Clase de grafuri - Drumurile P_n

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Drumul de ordin n , P_n : $V(P_n) = \{1, 2, \dots, n\}$ și $E(P_n) = \{12, 23, \dots, n-1n\}$.

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph



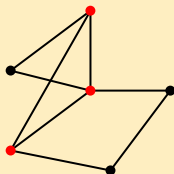
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

O submulțime de k noduri a graf G care induce un graf complet este numită o **k -clică**.

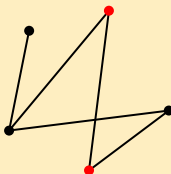
numărul de clică al lui G : $\omega(G) = \max_{Q \text{ clică în } G} |Q|$.

Remarcăm că $\omega(G) = \alpha(\overline{G})$.

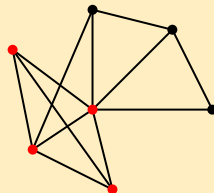
Exemplu



$$\omega(G) = 3$$



$$\omega(G) = 2$$



$$\omega(G) = 4$$

Clase de grafuri - Grafurile bipartite

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

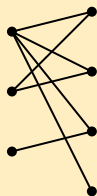
Graf bipartit: un graf G cu proprietatea că $V(G)$ poate fi partiționat în două clase care sunt mulțimi stabile.

Dacă $V(G) = S \cup T$, $S \cap T = \emptyset$, $S, T \neq \emptyset$, cu S și T mulțimi stabile în G , atunci G este notat $G = (S, T; E(G))$.

Graf bipartit complet: $G = (S, T; E(G))$, cu $uv \in E(G)$, $\forall u \in S$ și $\forall v \in T$; se notează cu $K_{s,t}$, unde $s = |S|$, $t = |T|$.

Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exemplu



A bipartite graph



$K_{1,1}$



$K_{1,2}$



$K_{2,3}$



$K_{3,3}$

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Graf planar: un graf care poate fi reprezentat într-un plan astfel ca fiecărui nod să îi corespundă un punct al acelui plan și fiecărei muchii să îi corespundă o curbă simplă care unește punctele corespunzătoare extremităților și **aceste curbe se intersectează doar în extremitățile lor**. Un graf care nu este planar este numit **graf ne-planar**.

Grafuri planare: **Problemă de decizie**

PLAN Instanță: G graf.

întrebare: Este G planar?

PLAN aparține clase de complexitate **P** (Hopcroft, Tarjan, 1972, $\mathcal{O}(n + m)$).

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

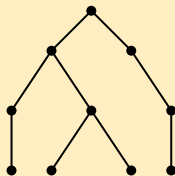
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Exemplu

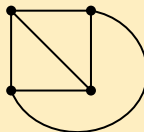
Grafuri planare.



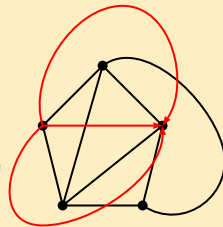
planar graph



planar graph



planar graph



K_5 non-planar graph

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

- Aceasta este metoda obișnuită de a defini o clasă de grafuri prin interzicerea unor anumite subgrafuri.
- Dacă $\mathcal{F}$ este o mulțime de grafuri atunci un graf G este **$\mathcal{F}$ -free** dacă G nu conține niciun subgraf indus izomorf cu vreun graf din $\mathcal{F}$.
- Dacă $\mathcal{F}$ este un singleton, $\mathcal{F} = \{H\}$, atunci scriem simplu **H -free**.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

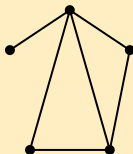
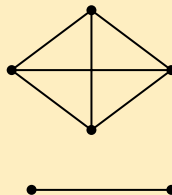
Exemplu

- clasa grafurilor nule este exact clasa grafurilor K_2 -free.
- Un graf P_3 -free este o reuniune disjunctă de grafuri complete.
- **Grafuri triangulate (cordale)**: grafurile $(C_k)_{k \geq 4}$ -free.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exemplu

Grafuri $\mathcal{F}$ -free.

a $2K_2$ -free grapha C_4 -free graph

a P_3 -free graph

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Fie $G = (V, E)$ un graf.

- Un **mers de lungime r** de la u până la v în G : o secvență de **noduri** și **muchii** de forma

$$(u =)v_0, v_0v_1, v_1, \dots, v_{r-1}, v_{r-1}v_r, v_r(=v).$$

u și v sunt extremitățile mersului.

- **Parcurs**: un mers cu muchii distincte.
- **Drum**: un mers cu noduri distincte.

Un nod este un mers (parcurs, drum) de lungime 0.

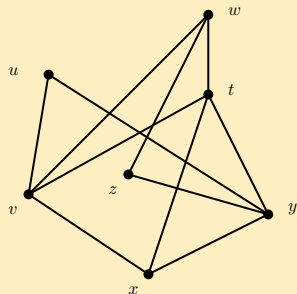
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Drumuri și circuite - Mersuri, parcursuri, drumuri

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exemplu

Mersuri, parcursuri, drumuri.



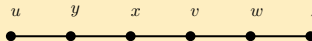
walk:



trail:



path:



Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Fie $G = (V, E)$ un graf.

- **Mers închis**: un mers de la u la u .
- **Circuit (drum închis)**: un mers cu noduri care sunt distincte cu excepția extremităților care coincid.
- Un **circuit** este **par** sau **impar** în funcție de paritatea lungimii sale.
- Lungimea celui mai scurt circuit (dacă există) este **grația**, $g(G)$, lui G .
- Lungimea celui mai lung circuit este **circumferința**, $c(G)$, lui G .

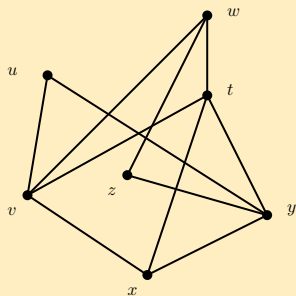
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Drumuri și circuite - Mersuri închise, circuite

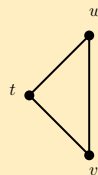
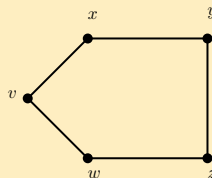
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exemplu

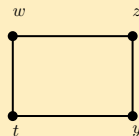
Mersuri închise, circuite.



odd circuits:



even circuit:

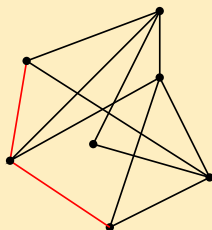


Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

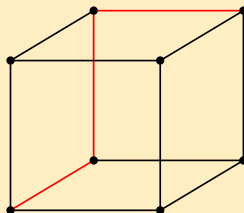
Fie $G = (V, E)$ un graf.

- **Distanța în G de la u la v , $d_G(u, v)$** lungimea celui mai scurt drum în G de la u la v (dacă există un astfel de drum).
- **Diametrul unui graf G , $d(G)$:**

$$d(G) = \max_{u, v \in V} d_G(u, v).$$



$$d(G) = 2$$



$$d(G) = 3$$

Fie $D = (V, E)$ un digraf.

Toate definițiile de mai sus se păstrează considerând **arce** (**muchii orientate**) în locul muchiilor.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.
Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

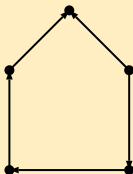
Fie $G = (V, E)$ un graf.

- **Graf conex**: există câte un drum între orice două noduri ale grafului. Altfel graful este **neconex**.
- **Componentă conexă** a unui graf G : un subgraf maximal conex, H , of G (i. e., nu există vreun subgraf conex H' of G , $H' \neq H$, iar H este subgraf al lui H').
- Orice graf poate fi scris ca o reuniune disjunctă a componentelor sale conexe.
- Următoarea relație binară este o relație de echivalență: $\rho \subseteq V \times V$, dată prin $u\rho v$ (i. e., $(u, v) \in \rho$) dacă există un drum în G între u și v .
- Componentele conexe ale lui G sunt subgrafurile induse de clasele de echivalență ale relației ρ .

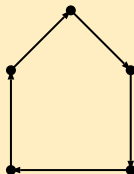
Fie $D = (V, E)$ un digraf.

- **Digraf slab conex** (sau simplu, **conex**) graful său suport $G(D)$ este conex.
- **Digraf unilateral conex**: există un drum de la u la v sau de la v la u , pentru orice două noduri $u, v \in V$.
- **Digraf tare conex**: există un drum de la u la v , pentru orice două noduri $u, v \in V$.

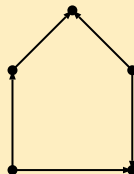
Exemplu



unilaterally connected



strongly connected



weakly connected

Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Fie $G = (V, E)$ un graf conex.

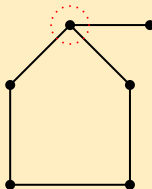
- **Punct de articulație (cut-vertex)**: un nod $v \in V$ astfel că $G - v$ este neconex.
- **Mulțime de articulație (vertex cutset)**: o mulțime de noduri $S \subseteq V$ așa încât $G - S$ este neconex.
- Un **arbore** este un graf conex fără circuite.
- Un graf ale cărui componente conexe sunt arbori este o **pădure**.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

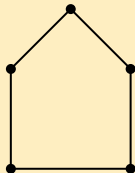
Mai general: într-un graf G care nu este neapărat conex $v \in V(G)$ este un **punct de articulație** dacă $G - v$ are mai multe componente conexe decât G .

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

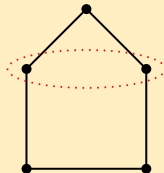
Exemplu



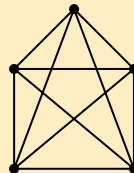
cut vertex



no cut vertex



vertex cutset



no vertex cutset

Fie $G = (V, E)$ un graf.

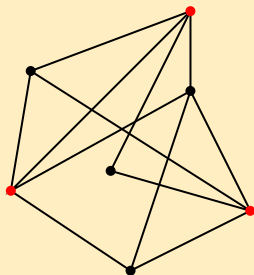
- Pentru $p \in \mathbb{N}^*$, G este **graf p -conex** dacă
 - ▶ $|V| = p$ și $G = K_p$ sau
 - ▶ $|V| \geq p + 1$ și G nu are mulțime de articulație de cardinal $< p$ (G nu poate fi deconectat prin ștergerea a mai puțin de p noduri).
- Evident, G este 1-conex dacă și numai dacă este conex.
- **Numărul de conexiune pe noduri**, $k(G)$, al unui graf G este

$$k(G) = \max \{p \in \mathbb{N}^* : G \text{ este } p\text{-conex}\}.$$

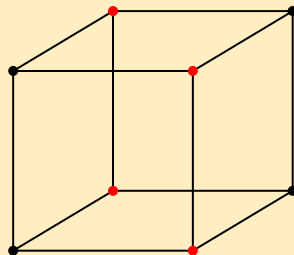
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Exemplu



$$k(G) = 3$$



$$k(G) = 4$$

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

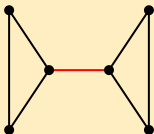
Fie $G = (V, E)$ un graf conex.

- **Punte (bridge)**: o muchie $e \in E$ astfel că $G - e$ nu este conex.
- **Mulțime de muchii de articulație (tăietură sau edge-cutset)**:
O submulțime de muchii $S \subseteq E$ așa încât $G - S$ este neconex.
- Pentru $p \in \mathbb{N}^*$, G este **graf p -muchie-conex** dacă G nu are o mulțime de muchii de articulație de cardinal $< p$ (G nu poate fi deconectat prin ștergerea a mai puțin de p muchii).
- **Numărul de conexiune pe muchii, $\lambda(G)$** , al unui graf G este

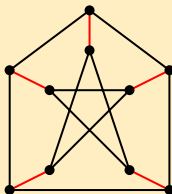
$$\lambda(G) = \max \{p \in \mathbb{N}^* : G \text{ este } p\text{-muchie-conex}\}.$$

Mai general: într-un graf G care nu este neapărat conex $e \in E(G)$ este o **punte** dacă $G - e$ are mai multe componente conexe decât G .

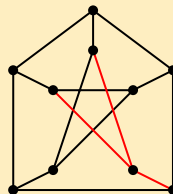
Exemplu



a cut edge



an edge-cutset


$$\lambda(G) = 3$$

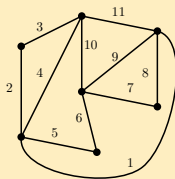
Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Drumuri și circuite - Grafuri Euleriene și Hamiltoniene

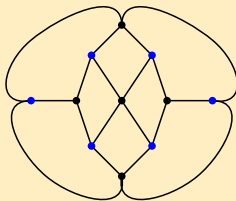
Fie G un (di)graf.

- G este **Eulerian** dacă există un parcurs închis în G care trece prin fiecare muchie a lui G .
- G este **Hamiltonian** dacă există un circuit în G care trece prin fiecare nod al lui G .

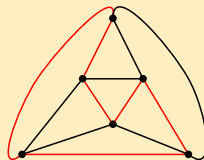
Recunoașterea (di)grafurilor Euleriene se face în timp polinomial (**Euler, 1736**).



an Eulerian graph



a non Hamiltonian graph
(bipartite of odd order)



a Hamiltonian graph

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Probleme Hamiltoniene

HAM Instanță: G un graf.

Întrebare: Este G Hamiltonian?

NP-completă (Karp, 1972).

NH Instanță: G un graf.

Întrebare: Este G ne-Hamiltonian?

NH $\in$ NP?

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții pentru seminarul de săptămâna viitoare

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.

Exercițiul 1. Fie G_1 și G_2 două grafuri. Arătați că dacă $G_1 \times G_2$ este conex, atunci G_1 și G_2 sunt conexe.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Exercițiul 2.

Pentru $k \in \mathbb{N}^*$, considerăm $G_k = K_2 \times K_2 \times \dots \times K_2$ (de k ori).

- (a) Determinați ordinul și dimensiunea lui G_k .
- (b) Arătați că G_k este bipartit și determinați $\alpha(G_k)$.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții pentru seminarul de săptămâna viitoare

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exercițiul 3.

Fie D un turneu conținând un circuit C de lungime $n \geq 4$. Arătați că pentru orice nod u al lui C se poate determina, în timpul $\mathcal{O}(n)$, un circuit de lungime 3 care trece prin u .

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.

Exercițiul 4.

Fie G un graf conex cu $n \geq 2$ noduri și m muchii. Arătați că:

- Dacă G are exact un circuit, atunci $m = n$.
- Dacă G nu are frunze, atunci $m \geq n$.
- Dacă G este arbore, atunci are cel puțin două frunze.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 5

Fie G un graf cu $n \geq 2$ noduri. Arătați că:

- a) Dacă G este conex, atunci conține cel puțin un nod care nu este punct de articulație.
- b) Dacă $n \geq 3$ și G este conex atunci conține două noduri care nu sunt puncte de articulație.
- (c) Adevărat sau fals: dacă G este conex iar x este un punct de articulație al său, atunci x este o frunză într-un arbore parțial al lui G ?

Exercițiul 6

Fie G un graf conex care nu conține două noduri pendante (frunze) cu un vecin în comun. Arătați că există două noduri adiacente prin ștergerea cărora G nu se deconectează.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exercițiul 7

Fie G un graf și H graful său reprezentativ al muchiilor ($H = L(G)$).
Arătați că H este $K_{1,3}$ -free.

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exercițiul 8

Fie G un graf. Arătați că:

- Dacă G are exact două noduri de grad impar, atunci aceste două noduri sunt unite printr-un drum în G .
- Dacă G este conex cu toate nodurile de grad par, atunci G are o muchie care nu este punte (ștergerea ei nu deconectează graful).
- Dacă G este conex cu toate nodurile de grad par, atunci nicio muchie a lui G nu este punte.

* Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Exercițiul 9

Fie G un graf. Arătați că

- Numărul de noduri de grad impar este par.
- Dacă G este conex și are k noduri de grad impar, atunci G este o reuniune $\lfloor k/2 \rfloor$ parcursuri disjuncte pe muchii.

Exercițiul 10

Fie G un graf astfel ca $N_G(u) \cup N_G(v) = V(G)$, $\forall u, v \in V(G)$, $u \neq v$. Arătați că G este graf complet.

Exercițiul 11

Fie G un graf cu proprietatea că $d_G(u) + d_G(v) \geq |G| - 1$, $\forall u, v \in V(G)$, $u \neq v$. Arătați că diametrul lui $d(G) \leq 2$.

Exercițiul for the next week seminar

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exercițiul 12

Fie $G = (V, E)$ un graf cu $V = \{v_1, v_2, \dots, v_n\}$ astfel ca $d_G(v_1) \leq d_G(v_2) \leq \dots \leq d_G(v_n)$. Arătați că G este conex dacă $d_G(v_p) \geq p$, pentru orice $p \leq n - d_G(v_n) - 1$.

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.

Exercițiul 13

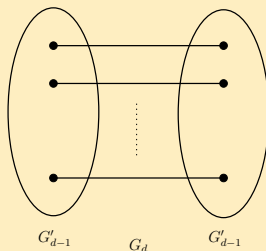
Fie $G = (V, E)$ un graf și S o mulțime stabilă a lui G . Demonstrați că S este stabilă de cardinal maxim dacă și numai dacă pentru orice mulțime stabilă a lui G , $S' \subseteq V \setminus S$, avem

$$|S'| \leq |N_G(S') \cap S|.$$

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 2. Soluție.

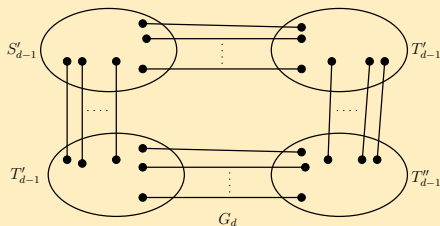
- (a) Deoarece pentru $d \geq 2$, $G_d = K_2 \times G_{d-1}$, putem construi G_d astfel: considerăm $G'_{d-1} \stackrel{\varphi}{\simeq} G''_{d-1}$ două copii isomorfe ale lui G_{d-1} și legăm perechile de noduri corespunzătoare (prin izomorfism) cu câte o muchie (vezi figura).



- Astfel $|G_d| = 2|G_{d-1}|$ and $|E(G_d)| = 2|E(G_{d-1})| + |G_{d-1}|$, cu $|G_1| = 2$ și $|E(G_1)| = 1$.
- Rezolvați aceste două recurențe.

(b) Demonstrăm prin inducție după d că G_d este bipartit cu bipartiția (S_d, T_d) și că $\alpha(G_d) = S_d = T_d = 2^{d-1}$.

- Afirmatia este evidentă pentru $d = 1$.
- În pasul inductiv presupunem că G_{d-1} ($k \geq 2$) este bipartit cu bipartiția (S_{d-1}, T_{d-1}) și $\alpha(G_{d-1}) = S_{d-1} = T_{d-1} = 2^{d-2}$ (vezi figura de mai jos).



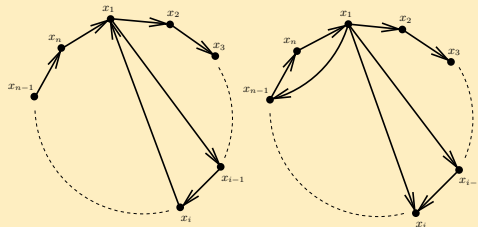
- Cum se poate defini o bipartiție pentru G_d folosind submulțimile $S_{d-1}, T_{d-1}, S'_{d-1}, T'_{d-1}$?

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Exercițiul 3. Soluție.

- Fie $V(C) = \{x_1, x_2, \dots, x_n\}$, $E(C) = \{x_1x_2, x_2x_3, \dots, x_nx_1\}$; presupunem că $x = x_1$. Parcurgem acest circuit: dacă $x_3x_1 \in E(D)$, returnăm $\{x_1, x_2, x_3\}$, altfel $x_1x_3 \in E(D)$; dacă $x_4x_1 \in E(D)$, atunci returnăm $\{x_1, x_3, x_4\}$ altfel ...
- Ce putem returna când toate corzile incidente cu x_1 pleacă spre exterior?



Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.

Algoritmul de complexitate $\mathcal{O}(n)$ este:

$i \leftarrow 3;$

while($x_1 x_i \in E(D)$)

$i++;$

if($i < n - 1$)

return $\{x_1, x_{i-1}, x_i\};$

else

return **what?;**

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Exercițiul 4. Soluție.

(a) $G - e$ este un arbore, $\forall e \in E(C)$ unde C este circuitul din G .

Puteți explica de ce?

(b) $2m = 2|E(G)| = \sum_{v \in V(G)} d_G(v) \geq 2|V(G)| = 2n.$

(c) Dacă G nu are frunze, atunci $m \geq n$ - contradicție.

- Dacă G are doar o frunză, atunci $2|E(G)| = \sum_{v \in V(G)} d_G(v) \geq 2|V(G)| - 1 \Rightarrow 2(|V(G)| - 1) \geq 2|V(G)| - 1$ - contradicție.

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

Exercițiul 5. Soluție.

- (b) Dacă T este un arbore parțial al lui G și $x \neq y$ sunt două frunze ale lui T , atunci $G - x$, $G - y$ sunt grafuri conexe. De ce?
- (c) Adevărat. Dar de ce?

Exercițiul 6. Soluție.

- Fie D un cel mai lung drum în G , $V(D) = \{x_1, x_2, \dots, x_p\}$;
- $N_G(x_1) \subseteq V(D) \setminus \{x_1\}$ altfel există drumuri mai lungi decât D . **De ce?**
- Presupunem că $G - \{x_1, x_2\}$ nu este conex, atunci există un nod $y \notin V(D)$ astfel ca $N_G(y) \cap V(D) = \{x_2\}$; mai mult, $d_G(y) = 1$. **De ce?**
- Repetând aceeași demonstrație cu $D - x_1 + y$ în loc de D obținem un alt nod z , cu $N_G(z) = \{x_2\}$ și $d_G(z) = 1$.
- Dar y și z sunt noduri pendante care au un același vecin (x_2) - contradicție.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Exercițiul 8. Soluție.

- (a) Dacă x și y (nodurile de grad impar) sunt în componente conexe ale lui G , atunci componenta care conține pe x are un singur nod de grad impar - contradicție. **Puteți explica de ce?** (Aveți în vedere suma tuturor gradelor.)
- (b) G are circuite (**de ce?**) iar o muchie de pe un circuit nu poate fi punte (**de ce?**).
- (c) Fie $xy \in E(G)$; x și y sunt singurele noduri de grad impar din $G - xy$, deci $G - xy$ este conex (i. e., xy nu este punte în G).

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exercițiul 9. Soluție.

- (a) $2|E(G)| = \sum_{v \in V(G)} d_G(v)$ iar un număr impar de noduri de grad impar ar face suma aceasta impară (!).
- (b) Adăugăm un nod nou la G și îl facem adiacent cu toate cele k noduri de grad impar; noul graf H are doar grade pare și este și conex, deci admite un parcurs eulerian închis. **Puteți spune de ce?**
- Ștergând nodurile adăugate obținem $k/2$ parcurhuri disjuncte pe muchii. (k este par iar muchiile șterse sunt în perechi de muchii adiacente - **de ce?**)

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 11. Soluție.

- Fie $u, v \in V(G)$, dacă $uv \notin E(G)$ și $N_G(u) \cap N_G(v) = \emptyset$, atunci $\{u, v\} \cap (N_G(u) \cup N_G(v)) = \emptyset$, adică $d_G(u) + d_G(v) \leq |G| - 2$ - contradicție.
- Astfel, dacă $u, v \in V(G)$ și $uv \notin E(G)$, atunci $N_G(u) \cap N_G(v) \neq \emptyset$ - de unde $d_G(u, v) = 2$. De ce?

Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 12. Soluție.

- Să presupunem prin reducere la absurd că G nu este conex.
- Fie G' o componentă conexă a lui G așa încât $v_n \notin V(G') = \{v_{j_1}, v_{j_2}, \dots, v_{j_p}\}$, $1 \leq j_1 < j_2 < \dots < j_p < n$.
- Deoarece G' nu conține v_n și nici vreun vecin de-al său, $p \leq n - d_G(v_n) - 1$. **De ce?**
- Astfel $d_G(v_p) \geq p$, dar $d_G(v_{j_p}) \geq d_G(v_p)$ și cum $\{v_{j_p}\} \cup N_G(v_{j_p}) \subseteq V(G')$, urmează că $p = |V(G')| \geq p + 1$ - contradicție.